Computer Security Symposium 2026
19-23 October 2026

ファジングにおける探索阻害要因の詳細な分類に向けた静的特徴の構成と分析

田久 健人[1,a)]　大月 勇人[1]　川古谷 裕平[1]　塩治 榮太朗[1]

概要：ソフトウェアの脆弱性発見に広く用いられるファジングでは，マジックバイトやチェックサムなどの探索阻害要因（fuzz blocker）により探索が停滞しうる．同種の fuzz blocker でも，値の生成過程や入力との依存関係によって攻略方法は異なりうるが，既知の種類だけではこうした違いを十分に区別できない．本稿では，fuzz blocker の詳細な分類に向け，攻略方法の違いを区別する静的特徴の候補を探索する．初期検討ではマジックバイトとチェックサムを対象とし，その攻略を支援する CmpLog に着目する．既存研究では比較処理の量を表す静的特徴と CmpLog の性能差に強い関係がみられないため，比較値の生成過程や入力との依存関係など，個々の比較処理の性質を表す属性を用いて静的特徴を新たに構成した．LLVM IR から特徴を抽出する解析器を実装し，FuzzBench と SBFT’23 のファジング標的 36 個に適用した．実験では，ハッシュ・チェックサム出力への依存など，CmpLog の性能差と関係する特徴がみられた．また，複数手法との比較や不一致事例の分析から，詳細な分類に向けて検証すべき特徴軸の候補を示した．

キーワード：ファジング，ソフトウェアテスト，静的解析

Construction and Analysis of Static Features Toward Fine-Grained Classification of Fuzz Blockers

Kento Takyu[1,a)]　Yuto Otsuki[1]　Yuhei Kawakoya[1]　Eitaro Shioji[1]

Abstract: Fuzzing is used for software vulnerability discovery, but exploration can stall due to fuzz blockers such as magic bytes and checksums. Even blockers of the same type may require different approaches depending on value generation and input dependencies, which existing categories do not distinguish. Toward fine-grained classification, we explore candidate static features that capture these differences. We focus on magic bytes and checksums and on CmpLog, which assists in overcoming them. Because prior work found no strong relationship between CmpLog performance and static features quantifying comparisons, we construct static features from attributes of comparisons, including value generation and input dependencies. We implement an LLVM IR analyzer and apply it to 36 targets from FuzzBench and SBFT’23. Our experiments reveal features associated with CmpLog performance, including dependence on hash and checksum outputs. Cross-technique comparisons and analyses of cases deviating from observed trends identify candidate feature axes for validation.

Keywords: Fuzzing, Software Testing, Static Analysis

1. はじめに

ファジングは，被検査プログラム（標的）に対するテスト入力の生成と実行を繰り返し，クラッシュ等の異常挙動を検出するソフトウェアテスト手法である [6]．ファジングでは，テスト入力を生成・変異しながら標的を実行し，入力に応じたプログラムの挙動や実行経路の変化を観測することで探索を進める．新たな実行経路に到達する入力を生成し，それまで未到達であったコード領域へ探索を広げ

[1] NTT 株式会社 社会情報研究所
Social Informatics Laboratories, NTT, Inc.
[a)] k.takyu@ntt.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

ることは，標的の多様な処理を検査する上で重要である．

一方，ファジングでは，いくつかの要因によって探索領域の拡大が妨げられることがある．例えば，素朴にランダムに入力を生成するだけでは，特定の値や複数の値の関係を要求する条件を通過できるテスト入力を生成することは難しいため，後続のコード領域への到達が困難となる．このようなファジングの探索を阻害する要因は fuzz blocker と呼ばれる [4]．本稿では，fuzz blocker によって妨げられている後続のコード領域へ到達できるようにすることを，fuzz blocker の攻略と呼ぶ．

Fuzz blocker は，特定の値との一致を要求するマジックバイトや，入力から算出した値との整合性を要求するチェックサムなど，いくつかの代表的な種類が知られている．また，それらの攻略を支援するため，CmpLog [1], [2] や TaintScope [8] など，さまざまな手法が提案されている．一方，同じ種類に含まれる fuzz blocker であっても，攻略方法が同じとは限らない．例えば，特定の定数値との一致を要求する処理でも，入力から得られた値を直接定数と比較する場合と，変換や計算を経た値を比較する場合では，攻略に必要な入力生成が異なる．このような攻略方法に関わる違いは，マジックバイトやチェックサムといった既知の種類だけでは十分に区別できない．そこで本研究では，fuzz blocker の詳細な分類に向け，攻略方法に関わる違いをプログラム上で観測可能な情報として表す静的特徴を検討する．

本稿では，その初期的な検討として，既知の fuzz blocker の種類のうちマジックバイトとチェックサム，およびそれらの攻略を支援する CmpLog に着目する．CmpLog は，実行時に観測した比較値を入力変異に利用する手法であり，その有効性には比較値の生成過程や入力との依存関係が関わると考えられる．

以下では，対象のファジング手法を導入した構成と基準構成とのカバレッジの差を表す指標を性能差と呼ぶ．プログラムの静的特徴とファジング手法の性能差との関係を分析した Zhang ら [9] は，CmpLog について比較処理に関する特徴と性能差との強い関係を確認できず，比較処理の量だけでは CmpLog の有効性を表せない可能性を指摘している．そこで本稿では，比較処理を一様に扱うのではなく，比較値の生成過程や入力との依存関係など，個々の比較処理の性質に着目する．

ただし，個々の比較処理が実際に fuzz blocker として探索を阻害するかを，静的解析のみから判定することは難しい．本稿では，特定の手法や試行条件ではなくプログラム側の性質に基づく特徴を検討するため，実行時の情報で候補を絞り込まず，比較処理に基づく条件分岐を fuzz blocker 候補として広く扱う．比較処理から取得した属性を標的単位に集約して静的特徴を構成する．攻略方法に関わる属性を持つ候補の現れ方が異なれば，CmpLog の相対性能にも系統的な差が現れる可能性があると考え，静的特徴と性能差との関係を分析する．この分析から個々の候補の攻略可否を直接判断することはできないが，関係がみられた静的特徴の基となる属性を，さらに検証すべき特徴軸の候補とする．

LLVM IR から静的特徴を抽出する解析器を実装し，FuzzBench [7] および SBFT'23 [5] の 36 標的に適用して，CmpLog の性能差との関係を分析した．さらに，データの揃う 23 標的で CmpLog 以外の手法との比較を行い，不一致事例についても分析した．

その結果，ハッシュ・チェックサム出力への依存を表す静的特徴は比較的一貫した正の相関を示した．また，入力由来値への依存が浅い比較の割合を表す静的特徴は負の相関を示したものの，ベンチマーク間で一貫しなかった．さらに，同じ静的特徴でも性能差との相関は手法によって異なり，不一致事例からは現在の静的特徴では区別できない性質が確認された．

本稿の貢献は以下の 3 点である．

- 比較値の生成過程や入力との依存関係など，個々の fuzz blocker 候補の性質を表す属性から静的特徴を新たに構成し，LLVM IR から抽出する解析器を実装した．
- 構成した静的特徴と CmpLog の性能差との関係を分析し，比較処理の量だけでは捉えにくい，ハッシュ・チェックサム出力への依存などの性質と性能差との関係を示した．また，CmpLog 以外の手法との比較から，これらの関係が手法によって異なることを示した．
- 不一致事例の分析から，比較値を入力へ直接反映するだけで条件を満たせるか，入力変異後に値の再生成が必要となるかなど，追加で検討すべき性質を整理した．

2. 背景と課題

2.1 ファジング

本稿では，ファジングの中でも，標的の実行から得られる情報を入力生成や探索に利用するグレーボックスファジングを主な対象とする．グレーボックスファジングでは，あらかじめ用意されたテスト入力や，それまでの探索で得られたテスト入力を変異して新たなテスト入力を生成し，標的を実行する．標的の実行から得られるコードカバレッジ等の情報は，その後の入力生成や探索にフィードバックされる．例えば，新たなコード領域に到達したテスト入力を保存し，その入力をさらに変異することで，より先の未到達領域への到達を目指す．

ファジングを実行するプログラムをファザーと呼ぶ．ファザーには，テスト入力の生成・変異，実行結果のフィードバックなど，探索を支援するさまざまな手法が組み込まれる．本稿では，これらの手法をファジング技法，または単に技法と呼ぶ．

© 2026 Information Processing Society of Japan

2.2 Fuzz blocker と攻略支援技法

Fuzz blocker とは，ファジングにおいて，後続のコード領域への到達を困難にし，プログラムの探索を妨げる要因である [4]．Gao ら [4] は，fuzz blocker を，探索を妨げる要因がテスト入力に依存するかに基づき，入力依存と入力非依存に大別している．

入力依存の fuzz blocker では，探索を妨げる要因がテスト入力の内容に依存するため，入力の生成や変異によって攻略を試みることができる．一方，入力非依存の fuzz blocker では，固定引数や必要な関数呼出しの欠如など，テスト入力を変化させるだけでは解消できない要因によって探索が妨げられる．本稿では，入力生成と攻略方法との関係を分析するため，入力依存の fuzz blocker に着目する．

また，入力依存の fuzz blocker として，入力中の特定の値との一致を要求するマジックバイトや，入力から算出された値と入力中の値との整合性を要求するチェックサムなどが知られている．このような fuzz blocker の攻略を支援する技法として，比較処理で観測された値を入力変異に利用する CmpLog [1], [2]，テイント解析を利用する TaintScope [8] など，さまざまなファジング技法が提案されている．

本稿では，これらに関わる比較処理について，実行時に観測した比較値を入力変異に利用する CmpLog に着目する．CmpLog は，REDQUEEN [1] で提案された手法に基づき，プログラムの実行中に比較処理のオペランドを記録し，観測した値を入力の変異に利用する手法である．例えば，マジックバイトに関わる処理では，入力から読み取った値と比較される定数値を入力へ反映することで，条件を満たせる場合がある．また，チェックサムに関わる処理では，入力から算出されたチェックサム値を入力中に格納されたチェックサム値へ反映することで，条件を満たせる場合がある．このように，CmpLog の有効性には，比較値の生成過程や入力との依存関係が関わると考えられる．

2.3 Fuzz blocker の詳細な分類に向けた課題

前節で述べたマジックバイトやチェックサムといった種類は，入力に要求される条件の違いを表す一方，同じ種類の内部にある攻略方法の違いまでは区別しない．例えば，比較値の生成過程や入力との依存関係が異なれば，攻略に必要な入力生成も異なる．

そこで本稿では，既知の種類に加えて，比較値の生成過程や入力との依存関係など，攻略方法に関わる性質によって同種の fuzz blocker をさらに区別することを，詳細な分類に向けた整理と位置付ける．

2.4 攻略方法に関わる性質を表す静的特徴

攻略方法に関わる違いは，比較値の生成過程や入力との依存関係，制御フロー上の位置，先行する条件との関係など，比較処理やその周辺のプログラム構造に反映される可能性がある．そこで本稿では，これらの違いを捉える手段として，プログラムの静的特徴に着目する．

プログラムの静的特徴とファジング技法の性能差との関係を分析した研究として，Zhang ら [9] の研究がある．Zhang らは，標的の規模や複雑度，制御構造，データ依存関係などの静的特徴と，基準構成に対する各ファジング技法の性能差との関係を分析している．

CmpLog についても，比較処理に関する静的特徴と基準構成に対する性能差との関係が分析されている．しかし，これらの特徴と CmpLog の性能差との間には強い関係が確認されていない．Zhang らはその理由として，全比較のうち CmpLog では攻略可能である一方，通常の入力変異では攻略が困難な比較が占める割合は小さく，比較処理の量だけでは CmpLog の有効性を表す指標にならない可能性を指摘している．

そこで本稿では，比較を一様に扱うのではなく，比較値の生成過程や入力との依存関係など，攻略方法に関係しうる比較処理間の違いを表す静的特徴を検討する．Zhang らが標的単位の静的特徴とファジング技法の性能差との関係を分析するのに対し，本稿では，性能差を手掛かりとして，個々の fuzz blocker 候補を攻略方法の違いに応じて記述するための特徴軸を探索する．

3. 静的特徴の構成

3.1 方針

本稿では，fuzz blocker の詳細な分類に向け，攻略方法に関係しうる比較処理の性質を捉える静的特徴を構成することを提案する．本章では，比較処理に基づく条件分岐を fuzz blocker 候補として広く扱い，そこから静的特徴を構成する方法を示す．個々の条件分岐が fuzz blocker として探索を阻害するかを静的解析のみから判定することは難しく，候補集合には実際には探索を阻害しないものも含まれうる．

各候補について，攻略方法に関係しうる性質を探索的に選び，比較形式，比較値の生成過程，入力との依存関係，制御フロー上の位置，先行する条件との関係などを表す属性を取得する．なお，入力依存性や制御フロー上の位置も分析対象とするため，候補抽出時には入力依存の有無や静的な到達可能性で絞り込まない．また，標的中の比較処理全体の性質と，fuzz blocker 候補となる条件分岐の性質とを区別して捉えるため，一部の属性は候補に限らず比較命令全体についても取得する．後の分析で用いる CmpLog の性能差が標的単位で得られるため，属性も標的単位に集約する．

3.2 比較処理から取得する属性

本稿では，値同士の等価関係や大小関係を判定する命令

© 2026 Information Processing Society of Japan

を比較命令と呼ぶ．また，比較命令や比較ルーチンによる比較結果に基づく条件分岐を比較ガードと呼ぶ．前節で述べた fuzz blocker 候補は，この比較ガードに対応する．

本稿では，入力に関する二つの起点を区別する．入力由来値は関数引数に由来するポインタから読み込まれた値とし，関数内の def-use 関係によるデータ依存解析の起点とする．このとき，関数境界を越えた入力の伝播は追跡しない．入力処理入口はファザーからテスト入力を受け取る処理の入口とし，比較ガードの制御フロー上の位置解析の起点とする．

個々の比較ガードまたは比較命令について取得する属性を**表 1** に示す．属性には，ある性質に該当するかを表す二値属性と，距離，個数などを表す数値属性がある．なお，MAGIC と HASHCONST は比較形式を表す属性であり，個々の比較が実際にマジックバイトやチェックサムに関する fuzz blocker であることを表すものではない．

INPUTDIST は入力由来値から比較値までの def-use 上の最短経路長，ENTRYDIST は，入力処理入口から比較ガードまでの，関数呼出しをまたいで接続した制御フローグラフ上の最短経路長である．INPUTDIST の集約では，入力由来値に依存する比較のみを対象とする．

HASHDEP は，ハッシュ・チェックサム出力への依存の有無に加え，依存する出力の数としても用いる．INPUTDIST2 は，入力由来値から比較値までの def-use 距離が小さい比較を区別するため，INPUTDIST が 2 以下であるかを表す．ここで，2 は分析上の操作的な閾値として設定する．なお，閾値を 4 に変更しても，後述する分析で得られる主要な傾向が変わらないことを確認した．

本稿では，先行する比較ガードを通らなければ後続の比較ガードへ到達できず（先行する比較ガードが後続を支配し），かつ後続の比較値が先行する比較ガードで用いられた値にデータ依存する場合，両者にガード間依存があるとする．GUARDDEP はその有無を，GUARDDEPCOUNT は依存する先行比較ガードの数を表す．

3.3 標的単位への集約

各属性を標的単位に集約して静的特徴を構成する．例えば，ある性質を持つ比較の個数と，集計対象の比較に占める割合は異なる情報を表す．また，数値属性についても，標的全体の傾向と，一部に極端な値を持つ比較が存在することは区別できる．そこで，各属性について，どの比較を対象とするかを表す集計領域と，その属性をどのように要約するかを表す集約方法を組み合わせる．これにより，同じ属性の標的中での現れ方を異なる観点から捉え，複数の静的特徴として表す．

本稿で用いる集計領域と集約方法を**表 2** に示す．集計領域には，CMP，GUARD，FAR を用いる．CMP は全比較命令を対象とし，標的中の比較処理全体の性質を捉えるための集計領域である．GUARD は全比較ガードを対象とし，fuzz blocker 候補全体の性質を捉えるための集計領域である．FAR は，当該比較へ至るまでにより多くの探索を要する可能性のある候補に着目するため，入力処理入口から手続き間制御フロー上で相対的に遠い比較ガードを対象とする．到達可能な比較ガードのうち，ENTRYDIST が上位 25%に含まれるものを FAR とする．ここで，25%は相対的に遠い比較ガードを抽出するための閾値として操作的に設定する．

属性，集計領域，集約方法を組み合わせて標的単位の静的特徴を構成し，「属性_集計領域_集約方法」の順に名前を付ける．例えば，HASHDEP_GUARD_FRAC は，ハッシュ・チェックサム出力に依存する比較ガードが全比較ガードに占める割合を表す．

二値属性には主に COUNT または FRAC を，数値属性には MEAN または MAX を適用する．属性ごとに適用可能な集計領域と集約方法を探索的に組み合わせ，値が自明に定まる組合せを除いた計 51 個の静的特徴を構成する．

4. 実験

4.1 実験の目的と研究設問

本章では，3 章で構成した標的単位の静的特徴と，CmpLog の有無による性能差との関係を分析する．ある属性が fuzz blocker の攻略方法に関係するなら，その属性を持つ候補の現れ方が異なる標的間では，CmpLog の相対性能にも系統的な違いが現れる可能性がある．性能差の分析を通じて，fuzz blocker の詳細な分類に向けて，個々の候補についてさらに検証すべき特徴軸を探索する．本稿では，相関の大きさや傾向の一貫性，技法の動作との整合性を手掛かりとして，今後検証すべき属性を探索的に選ぶ．次の三つの研究設問に基づいて調べる．

RQ1 構成した静的特徴は，CmpLog の有無による性能差とどのような関係を示すか．

RQ2 RQ1 で着目した静的特徴と性能差との関係は，ファジング技法によってどのように異なるか．

RQ3 RQ1 で得られた傾向と実測結果が一致しない事例には，現在の静的特徴では捉えられていないどのような性質がみられるか．

4.2 実験設定

ファジングの実測データには，Zhang ら [9] が公開した実験データを用いる．分析対象は，FuzzBench の 23 標的と SBFT'23 の 13 標的の計 36 標的である．このうち，公開データでは実測値が得られない実験条件と標的の 6 組については，本稿で新たに実験を行い，各組 20 試行の結果を取得して補完する．補完実験のうち一部の標的では公開データと本稿の測定値に水準差が確認されたため，当該標的では本稿のパイプラインで再取得した基準構成（naive

© 2026 Information Processing Society of Japan

表 1　静的特徴を構成する属性とその意味.

Table 1　Attributes used to construct static features and their meanings.

観点	属性	型	意味
比較形式	MAGIC	二値	HASHCONST 以外の非ゼロ定数との等価比較であるか
	HASHCONST	二値	ハッシュ・チェックサム出力に依存する値と定数との等価比較であるか
	OTHER	二値	上記以外の比較であるか
値の生成・依存	HASHDEP	二値・数値	ハッシュ・チェックサム出力への依存の有無・依存する出力数
	INPUTDEP	二値	入力由来値へのデータ依存の有無
	INPUTDIST	数値	入力由来値から比較値までの def-use グラフ上の最短経路長
	INPUTDIST2	二値	入力由来値に依存し，かつ入力由来値への依存が浅い（INPUTDIST ≤ 2）か
	WIDTH	数値	比較対象の大きさ（整数比較ではビット幅）
制御フロー上の位置	ENTRYDIST	数値	入力処理入口から比較ガードまでの手続き間制御フローグラフ上の最短経路長
	FARFROMENTRY	二値	到達可能な比較ガードのうち，ENTRYDIST が上位 25%に含まれるか
ガード間の関係	GUARDDEP	二値	先行比較ガードとのガード間依存の有無
	GUARDDEPCOUNT	数値	ガード間依存する先行比較ガードの数

表 2　集計領域と集約方法.

Table 2　Aggregation regions and aggregation operators.

種類	記号	意味	捉える側面
集計領域	CMP	全比較命令	比較処理全体
	GUARD	全比較ガード	比較ガード全体
	FAR	入力処理入口から遠い比較ガード	入力処理入口から遠い比較ガード
集約方法	COUNT	個数	絶対的な多さ
	FRAC	割合	集計領域に占める相対的な多さ
	MEAN	平均	属性の全体的な傾向
	MAX	最大値	一部に存在する大きな値

条件）を基準とした．詳細は付録 A.3 に示す．

各試行は，FuzzBench の標準的な時間予算である 23 時間実行され，終了時の被覆分岐数を性能指標に用いる．各標的について，対象技法を導入した条件と naive 条件のカバレッジの分布から標準化平均差である Cohen の d を求め，これを性能差とする．$d > 0$ は，対象技法を導入した条件が naive 条件より有利であることを表す．

RQ1 では，共通のファジングフレームワーク LibAFL [3] 基盤に CmpLog を導入した CmpLog 条件と，導入せずに同基盤をそのまま用いる naive 条件を比較する．36 標的について，両条件を 20 試行ずつ用いる．

RQ2 では，Zhang ら [9] が評価した 12 実験条件，すなわち，変異方法を変更する技法（CmpLog，grimoire，mopt），フィードバックを変更する技法（value profile，ngram4，ngram8，ctx），スケジューリングを変更する技法（explore，fast，cov accounting，rand scheduler，weighted）を対象とし，36 標的のうち，すべての条件のデータが得られる共通 23 標的を用いる．各実験条件は，naive 条件に特定のファジング技法を追加または差し替えた構成であり，naive 条件に対する Cohen の d を性能差として求める．

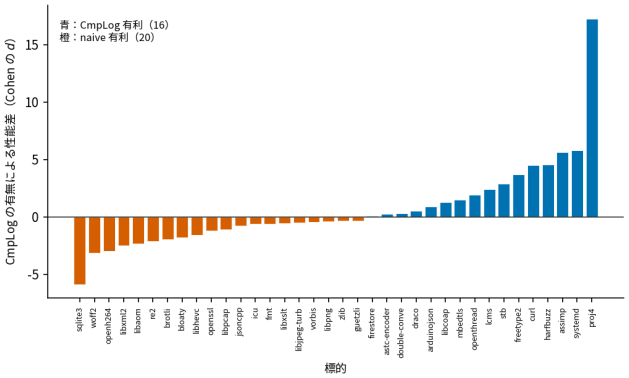


図 1　全 36 標的における CmpLog 条件と naive 条件の性能差.

Fig. 1　Performance differences between the CmpLog and naive conditions across all 36 targets.

RQ3 では，RQ1 の結果に基づいて，静的特徴から予想される傾向と実測結果が一致しない標的を抽出し，比較処理とその周辺のソースコードを調査する．

図 1 に，RQ1 で用いる 36 標的における CmpLog 条件と naive 条件の性能差を示す．性能差は $d = -5.9$ から $+17.3$ まで標的によって大きく異なり，CmpLog 条件が有利な標的と naive 条件が有利な標的の双方が存在する．

静的特徴は，3 章の構成に従って LLVM 15 IR から抽出する．データ依存の解析では関数内の def-use 関係を用い，メモリを介した値の流れを近似的に扱う．また，ハッシュ・チェックサム出力は既知の関数名に基づいて識別する．解析器の実装の詳細は付録 A.2 に示す．

4.3　分析方法

静的特徴と性能差の単調な関係を調べるため，Spearman 順位相関係数 ρ を用いる．$\rho > 0$ は特徴が大きい標的ほど対象技法が naive 条件に対して相対的に有利となる傾向を，$\rho < 0$ はその逆を表す．

© 2026 Information Processing Society of Japan

FuzzBench と SBFT'23 では，静的特徴や性能差の水準が異なる可能性がある．そこで，両ベンチマークを統合した全 36 標的の相関に加えて，各ベンチマーク内の相関も求め，傾向がベンチマーク間で一貫するかを確認する．また，比較ガード数の多い標的では各特徴の個数も大きくなりやすいため，比較ガード数を制御した偏相関も求める．

4.4 RQ1：静的特徴と CmpLog の性能差との相関

まず，既存研究で用いられた，主にプログラムの規模や複雑さを表す静的特徴と，CmpLog の有無による性能差との相関を確認する．Zhang ら [9] が用いた静的特徴について本稿の 36 標的で相関を求めた．分析に用いられた比較命令数，基本ブロック数，分岐数などと性能差との相関は全体として弱く，例えば，比較命令数との Spearman 順位相関係数は $\rho = +0.21$ であった．この傾向は，比較処理に関する静的特徴と CmpLog の性能差との強い相関は確認されなかったとする Zhang らの報告と整合する．

次に，3 章で構成した，比較値の生成過程や入力由来値との依存関係などを表す静的特徴と CmpLog の性能差との相関を分析する．**図 2** に，本稿で構成した静的特徴と CmpLog の性能差との Spearman 順位相関係数 ρ を示す．縦軸は静的特徴，横軸は各静的特徴と CmpLog の性能差との Spearman 順位相関係数 ρ を表す．図中では，ハッシュ・チェックサム出力への依存に関する特徴を青，入力由来値への依存に関する特徴を橙，その他の特徴を灰色で示す．なお，全 51 静的特徴の相関係数を付録 A.4 に示す．

ハッシュ・チェックサム出力への依存に関する特徴では，CmpLog の性能差との正の相関がみられた．例えば，HASHDEP_GUARD_FRAC の相関係数は $\rho = +0.46$ であり，FuzzBench と SBFT'23 でもそれぞれ +0.55，+0.30 と正の値を示した．また，HASHCONST に関する特徴でも同様に正の相関がみられた．これらの傾向は，比較ガード数を制御した偏相関でも維持された．

一方，入力由来値への依存が浅い比較の割合を表す INPUTDIST2 に関する特徴では，負の相関がみられた．例えば，INPUTDIST2_CMP_FRAC は $\rho = -0.39$，INPUTDIST2_GUARD_FRAC は $\rho = -0.36$ であった．ただし，INPUTDIST2_CMP_FRAC は FuzzBench で $\rho = -0.37$，SBFT'23 で $\rho = -0.01$ であり，傾向はベンチマーク間で一貫しなかった．この相関は，浅い依存が CmpLog を阻害することを意味しない．naive 条件でも容易に攻略可能な比較が多いことや，標的単位の集約による交絡が考えられるため，機構的な解釈には追加分析が必要である．

RQ1 の答え． ハッシュ・チェックサム出力への依存を表す静的特徴は，CmpLog の性能差と正の相関を示し，両ベンチマークで同方向の相関を示した．一方，入力由来値への依存が浅い比較の割合を表す静的特徴は負の相関を示したが，この傾向はベンチマーク間で一貫しなかった．

4.5 RQ2：他のファジング技法との比較

RQ2 では，RQ1 で性能差との関係がみられた属性のうち，ハッシュ・チェックサム出力への依存を表す HASHDEP と，入力依存系で比較的大きな負の相関を示した INPUTDIST2 を取り上げ，技法間で比較する．

4.2 節の 12 実験条件・共通 23 標的について，naive 条件との性能差と各静的特徴との Spearman 順位相関係数 ρ を求める．集計領域による違いも調べるため，HASHDEP と INPUTDIST2 について GUARD と FAR を用い，集約方法を FRAC に揃えた 4 特徴を比較する．

図 3 に，RQ1 で着目した属性に基づく静的特徴と各実験条件の naive 条件に対する性能差との Spearman 順位相関係数 ρ を示す．各セルの値は，当該静的特徴と naive 条件に対する性能差との相関係数 ρ であり，赤は正の相関，青は負の相関を表す．なお，全 51 静的特徴と各実験条件との相関を付録 A.5 に示す．

まず，ハッシュ・チェックサム出力への依存を表す HASHDEP_GUARD_FRAC は，CmpLog で +0.55，cov accounting で +0.42，rand scheduler で +0.51 と正の相関を示した．一方，grimoire では +0.16 であり，相関の大きさは技法によって異なった．なお，RQ2 は共通 23 標的に限定するため，CmpLog の相関係数も RQ1 の 36 標的での値とは異なる．

入力由来値への浅い依存を表す INPUTDIST2_GUARD_FRAC は，CmpLog と grimoire でそれぞれ −0.40，−0.51 と負の相関を示した．一方，ngram4 では +0.30 と正の相関を示し，同じ特徴でも相関の方向が技法によって異なった．

さらに，同じ属性でも集計領域によって技法横断の相関パターンが異なった．HASHDEP_FAR_FRAC は，HASHDEP_GUARD_FRAC で相関の小さかった grimoire でも +0.46 となった一方，ngram4 と ngram8 ではそれぞれ −0.33，−0.45 となった．このように，同じ属性であっても，比較の入力処理入口からの制御フロー上の位置によって，技法横断の相関パターンに違いがみられた．

一つの解釈として，FAR に含まれる比較では手前の条件を通過する必要があるため，当該比較の性質だけでなく，そこまでの探索を各技法がどのように支援するかによって，性能差との関係が変化すると考えられる．

RQ2 の答え． RQ1 で着目した静的特徴は，CmpLog 以外の複数のファジング技法の性能差とも相関を示した．ただし，相関の方向や大きさは技法や集計領域によって異なった．これらの静的特徴と性能差との関係は，特定の技法と一対一に対応しない．

4.6 RQ3：現在の静的特徴では説明しにくい事例の分析

RQ1 では，HASHDEP_GUARD_FRAC（以降，簡単のため $\text{HASHDEP}_{\text{F}}$ と記す）が大きい標的ほど CmpLog が相

© 2026 Information Processing Society of Japan

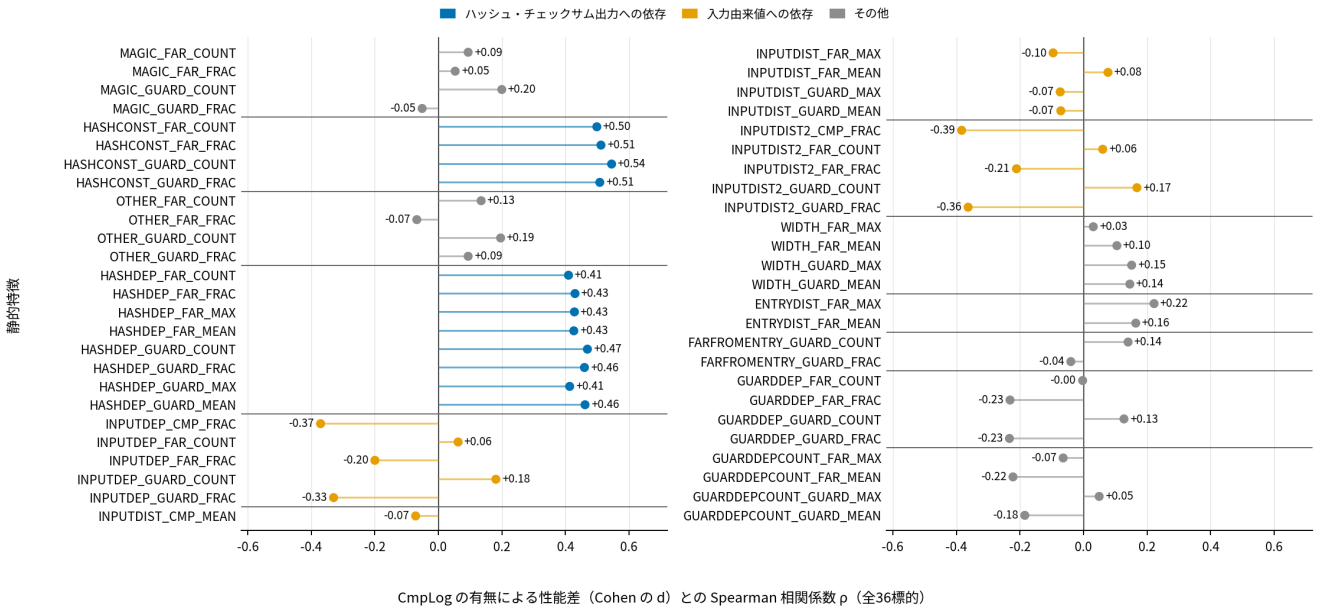


図 2　構成規則から生成した全 51 静的特徴と CmpLog の有無による性能差との相関.

Fig. 2　Correlations between all 51 static features and the performance difference between configurations with and without CmpLog.

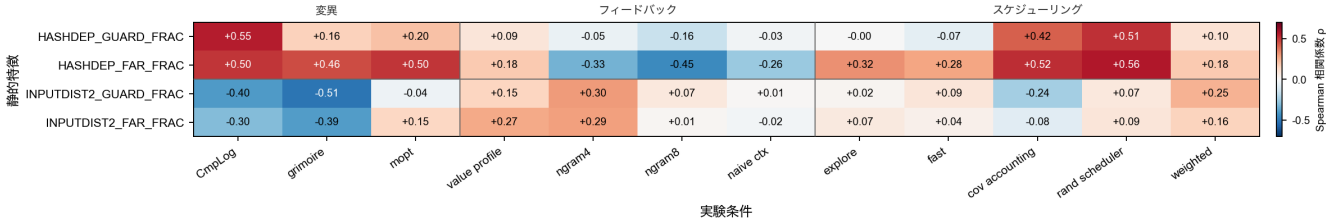


図 3　各静的特徴と各実験条件の naive 条件に対する性能差との相関.

Fig. 3　Correlations between static features and the performance difference of each experimental condition relative to the naive condition.

対的に有利となる傾向がみられた．一方，この傾向だけでは説明できない標的も存在する．このような事例には，$\mathrm{HASHDEP_F}$ だけでは捉えられない fuzz blocker 候補の性質が含まれている可能性がある．そこで，$\mathrm{HASHDEP_F}$ から予想される傾向と実測結果が一致しない標的を抽出し，比較処理とその周辺のソースコードを調査する．

各ベンチマーク内で，$\mathrm{HASHDEP_F}$ が上位四分位かつ $d \leq 0$ の標的と，$\mathrm{HASHDEP_F}$ が下位四分位かつ d が上位四分位の標的を抽出した．高 $\mathrm{HASHDEP_F}$ 側では，HASHDEP に該当する比較に CmpLog の性能差と関係しうる別の性質がないかを，低 $\mathrm{HASHDEP_F}$ 側では，HASHDEP 以外に CmpLog による攻略対象となりうる比較の性質がないかを調べるため，異なる基準を用いた．

該当した zlib, openssl, libjpeg-turbo, stb, arduinojson, libcoap の 6 標的について，高 $\mathrm{HASHDEP_F}$ 側では静的解析器が HASHDEP として計上した比較を，低 $\mathrm{HASHDEP_F}$ 側では HASHDEP に計上されない比較を調査対象とした．解析器が出力した所在関数名を手掛かりに，ソースコード上の該当箇所と周辺の入力処理を確認した．RQ3 では現在の静的特徴で捉えられていない性質を探索するため，調査観点を事前に固定せず，本節で示す性質はコード調査後に整理した．各標的では代表的な比較箇所を対象とする探索的ケーススタディとし，到達可能性と比較形式の分布については解析器出力に基づく全数集計も用いた．

表 3 に，$\mathrm{HASHDEP_F}$ から予想される傾向と実測結果が一致しない標的と，コード調査でみられた性質を示す．以降，観測した比較値を入力へ直接反映するだけで条件を満たせる場合を書戻し可能と呼び，その可否を書戻し可能性とする．

表 3 上段に示す高 $\mathrm{HASHDEP_F}$・CmpLog 非有利の標的では，HASHDEP だけでは区別できない性質として，書戻し可能性，変異後の再生成の要否，入力処理入口からの到達可能性がみられた．openssl では書戻しだけでは条件を満たせない処理が含まれた．zlib では入力変異後にチェックサムの再生成が必要であった．また，libjpeg-turbo では，HASHDEP として計上された比較の一部が入力処理入

© 2026 Information Processing Society of Japan

表 3 $\mathsf{HASHDEP_F}$ から予想される傾向と実測結果が一致しない標的と，不足する性質の候補．

Table 3 Targets whose observed performance does not match the tendency expected from $\mathsf{HASHDEP_F}$, and candidate properties not captured by the current features, as identified through code inspection.

不一致の分類	標的	不足する性質の候補
高 $\mathsf{HASHDEP_F}$・CmpLog 非有利	zlib・openssl・libjpeg-turbo	書戻し可能性，変異後の再生成の要否，入力処理入口からの到達可能性
低 $\mathsf{HASHDEP_F}$・CmpLog 有利	stb・arduinojson・libcoap	HASHDEP に該当しない書戻し可能な比較の存在，複数の条件の組合せ

口から到達できないコードに含まれていた．到達可能性自体は ENTRYDIST の算出過程で判定しているが，現在の静的特徴は到達不能な候補の割合を独立した属性として集約していない．

一方，表 3 下段に示す低 $\mathsf{HASHDEP_F}$・CmpLog 有利の標的では，ハッシュやチェックサム出力への依存以外にも CmpLog が支援可能な比較がみられた．stb では書戻し可能な複数バイトの定数比較が多数存在した．arduinojson と libcoap では複数の比較条件を満たす必要がある箇所がみられた．

なお，GUARDDEP は支配関係とデータ依存を同時に満たすガード間関係のみを捉えるため，互いにデータ依存のない複数のフィールドを同時に満たす必要がある場合などは表現できない．

RQ3 の答え． 不一致事例の探索的なコード調査からは，値の書戻し可能性，変異後の再生成の要否，入力処理入口からの到達可能性，複数条件の組合せなどの違いがみられた．

5. 議論

5.1 Fuzz blocker の詳細な分類に向けた示唆

RQ1・RQ2 で性能差との関係がみられた静的特徴の基となる属性は，特定のファジング技法に固有の適用条件とは言えない一方，個々の fuzz blocker 候補について今後検証すべき特徴軸の候補と位置付けられる．

RQ3 の探索的ケーススタディでは，書戻し可能性，変異後の再生成の要否，入力処理入口からの到達可能性，複数条件の組合せなど，HASHDEP だけでは区別できない性質の候補がみられた．以上から，fuzz blocker の詳細な分類には，攻略方法に関わる複数の性質を組み合わせて記述する必要があると考える．

5.2 制約

本稿では多数の静的特徴と性能差との関係を探索的に分析しており，個々の相関は仮説検証を目的とした確証的な結果ではない．また，静的特徴と性能差はいずれも標的単位で集約しているため，個々の fuzz blocker 候補の性質と攻略結果を直接対応付けたものではない．候補集合には実際には探索を阻害しない比較や，入力処理入口から到達不能な比較も含まれうる．評価対象は 36 標的および LibAFL 上の実験条件に限られ，一部の傾向は FuzzBench と SBFT'23 で一貫しないため，他の標的や実験条件への一般化には追加検証が必要である．さらに，本稿はマジックバイトとチェックサムを中心とする初期検討であり，入力由来値を関数引数由来のポインタからの load で近似するなど，静的特徴の抽出には近似や誤検出が含まれうる．

6. おわりに

本稿では，比較処理に基づく fuzz blocker 候補から静的特徴を構成し，CmpLog の性能差との関係を分析した．ハッシュ・チェックサム出力への依存では比較的一貫した正の相関がみられ，入力由来値への依存の浅さでは負の相関がみられたものの，後者はベンチマーク間で一貫しなかった．また，相関の傾向はファジング技法によって異なり，不一致事例からは追加で検討すべき性質の候補が得られた．

以上から，性能差との関係がみられた静的特徴の基となる属性を，fuzz blocker 単位で今後検証すべき特徴軸の候補として整理した．今後は，候補ごとの属性と攻略結果を比較ガード単位で対応付け，特徴軸の妥当性を検証する．

参考文献

[1] Cornelius Aschermann, Sergej Schumilo, Tim Blazytko, Robert Gawlik, and Thorsten Holz. REDQUEEN: Fuzzing with input-to-state correspondence. In *Proc. NDSS*, 2019.

[2] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++: Combining incremental steps of fuzzing research. In *Proc. USENIX WOOT*, 2020.

[3] Andrea Fioraldi, Dominik Maier, Dongjia Zhang, and Davide Balzarotti. LibAFL: A framework to build modular and reusable fuzzers. In *Proc. ACM CCS*, 2022.

[4] Wentao Gao, Van-Thuan Pham, Dongge Liu, Oliver Chang, Toby Murray, and Benjamin I. P. Rubinstein. Beyond the coverage plateau: A comprehensive study of fuzz blockers (registered report). In *Proc. FUZZING*, pages 47–55, 2023.

[5] Dongge Liu, Jonathan Metzman, Marcel Böhme, Oliver Chang, and Abhishek Arya. SBFT tool competition 2023 – fuzzing track. In *Proc. IEEE/ACM SBFT*, pages 51–54, 2023.

[6] Valentin J.M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. The art, science, and engineering of fuzzing: A survey. *IEEE Trans. Software Engineering*, 47(11):2312–2331, 2021.

[7] Jonathan Metzman, László Szekeres, Laurent Simon, Read Sprabery, and Abhishek Arya. FuzzBench: An open fuzzer benchmarking platform and service. In *Proc. ACM ESEC/FSE*, 2021.

[8] Tielei Wang, Tao Wei, Guofei Gu, and Wei Zou. TaintScope: A checksum-aware directed fuzzing tool for automatic software vulnerability detection. In *Proc. IEEE S&P*, 2010.

[9] Dongjia Zhang, Andrea Fioraldi, and Davide Balzarotti. On understanding and forecasting fuzzers performance with static analysis. In *Proc. ACM CCS*, 2024.

© 2026 Information Processing Society of Japan

付　　録

A.1　正式な定義と静的特徴の構成

本付録では，3 章で用いた比較ガード，データ依存，入力処理入口からの距離，ガード間依存，および静的特徴の構成を定義する．

A.1.1　解析に用いるグラフ

プログラム P の各関数について，次のグラフと関係を用いる．

- **制御フローグラフ（CFG）**：基本ブロックを頂点，制御フローを辺とする有向グラフ $G_{\mathrm{cfg}} = (N, E)$．
- **支配関係**：基本ブロック n が m を支配するとは，関数入口から m へのすべての経路が n を通ることをいう．
- **def-use グラフ**：命令を頂点とし，SSA 形式上の def-use 関係を辺とする有向グラフ $D = (V_D, E_D)$．メモリを介した値の流れを近似するため，同一の基底オブジェクトに対する store と load の間にも辺を追加する．関数境界を越える def-use 関係は追跡しない．

本稿では，データ依存解析に用いる入力由来値と，制御フロー上の位置の解析に用いる入力処理入口を区別する．本稿では，`LLVMFuzzerTestOneInput` の入口を入力処理入口とし，手続き間制御フローグラフ上で各比較ガードまでの距離を求める．以下では，両者を区別して定義する．

A.1.2　比較ガードと比較形式

定義 A.1.1 (比較命令と比較ガード)．値同士の等価関係や大小関係を判定する命令を比較命令と呼ぶ．比較命令または比較ルーチンによる比較結果に基づく条件分岐を比較ガードと呼び，プログラム P 中の比較ガードの集合を G とする．解析対象には，`icmp` 等の比較命令による分岐と，`memcmp` や `strcmp` 等の比較ルーチンの結果による分岐を含める．各 $g \in G$ について，その位置を $\mathrm{loc}(g)$，比較幅を $m(g)$ とする．

定義 A.1.2 (比較形式)．比較ガード g の比較形式 $\tau(g)$ を，

$$\tau(g) = \begin{cases} \mathsf{HashConst} & g \text{ が定数等価比較かつ} \\ & \quad \exists h \in H,\ g \Leftarrow_D h, \\ \mathsf{Magic} & g \text{ が非ゼロ定数との等価比較}, \\ \mathsf{Other} & \text{otherwise} \end{cases}$$

とする．ただし，上から順に判定する．

ゼロ定数との等価比較は，null 検査やエラー判定等にも広く現れるため，Magic には含めない．HashConst は，暗号学的ハッシュに限らず，チェックサム，CRC，MAC 等の出力への依存を含む．

A.1.3　比較値の生成過程と入力由来値への依存

定義 A.1.3 (データ依存)．比較ガードまたは比較命令を x とし，def-use グラフ $D = (V_D, E_D)$ 上で，x の非定数オペランドを生成する命令の集合を $\mathrm{Def}(x) \subseteq V_D$ とする．命令 $v \in V_D$ が $\mathrm{Def}(x)$ のいずれかへ到達できるとき，x は v にデータ依存するといい，$x \Leftarrow_D v$ と書く：

$$x \Leftarrow_D v \iff \exists r \in \mathrm{Def}(x),\ v \xrightarrow{*}_D r$$

また，v から $\mathrm{Def}(x)$ までの最短距離を

$$\delta_D(x, v) = \min_{r \in \mathrm{Def}(x)} \mathrm{dist}_D(v, r)$$

とする．

入力由来値を生成する命令の集合を I_{val}，ハッシュやチェックサムの出力を生成する命令の集合を H とする．I_{val} は各関数において引数に由来するポインタからの load 命令，H は指定したハッシュ・チェックサム関数の戻り値を生成する命令とする．解析は関数内で行うため，I_{val} は入力バッファ由来の値の近似である．

定義 A.1.4 (ハッシュ・チェックサム出力への依存)．比較ガード g について，

$$\mathrm{hashdep}(g) = |\{h \in H \mid g \Leftarrow_D h\}|$$

とする．$\mathrm{hashdep}(g) > 0$ であれば，g の比較値はハッシュまたはチェックサムの出力に依存する．この依存の有無および個数を属性 HASHDEP として用いる．

定義 A.1.5 (入力由来値への依存と距離)．比較ガードまたは比較命令 x について，入力由来値から比較値までの最短 def-use 距離を

$$d_{\mathrm{input}}(x) = \min_{i \in I_{\mathrm{val}}} \delta_D(x, i)$$

とする．入力由来値に依存しない場合は $d_{\mathrm{input}}(x) = -1$ とする．$d_{\mathrm{input}}(x) \geq 0$ であるかを属性 INPUTDEP として用いる．また，$d_{\mathrm{input}}(x) \geq 0$ を満たす比較については，$d_{\mathrm{input}}(x)$ を属性 INPUTDIST として用いる．また，本稿では

$$0 \leq d_{\mathrm{input}}(x) \leq 2$$

であるかを属性 INPUTDIST2 とし，入力由来値から少数の処理を介して生成された比較値を区別する．

A.1.4　入力処理入口からの制御フロー上の位置

本稿では，ファザーからテスト入力を受け取る関数 `LLVMFuzzerTestOneInput` の入口を入力処理入口とする．入力処理入口は，前節の入力由来値とは異なり，比較ガードの制御フロー上の位置を求めるための起点である．また，各関数の制御フローグラフを関数呼出しの辺で接続したグラフを手続き間制御フローグラフと呼ぶ．比較ガード g の制御フロー上の位置を表すため，入力処理入口から g

© 2026 Information Processing Society of Japan

までの距離を定義する．

定義 A.1.6 (入力処理入口からの距離)．入力処理入口から比較ガード g までの手続き間制御フローグラフ上の最短経路長を $d_{\mathrm{entry}}(g)$ とする．入力処理入口から到達できない場合は $d_{\mathrm{entry}}(g) = \infty$ とする．この距離を属性 ENTRYDIST として用いる．

定義 A.1.7 (入力処理入口から遠い比較ガード)．入力処理入口から到達可能な比較ガードのうち，$d_{\mathrm{entry}}(g)$ が上位25%に含まれる比較ガードの集合を B_{far} とする．比較ガード g が B_{far} に含まれるかを属性 FARFROMENTRY として用いる．

A.1.5 先行比較ガードとのガード間依存

定義 A.1.8 (ガード間依存)．比較ガード g の位置を支配する比較ガードの集合を

$$\mathrm{PredGuard}(g) = \{h \in G \backslash \{g\} \mid \mathrm{loc}(h) \text{が} \mathrm{loc}(g) \text{を支配する}\}$$

とする．g が，$h \in \mathrm{PredGuard}(g)$ の非定数オペランドを生成する命令のいずれかにデータ依存するとき（$\exists v \in \mathrm{Def}(h),\ g \Leftarrow_D v$），$g$ と h の間にガード間依存があるとする．g とガード間依存がある先行比較ガードの数を $\mathrm{guarddep}(g)$ とする．$\mathrm{guarddep}(g) > 0$ であるかを属性 GUARDDEP，$\mathrm{guarddep}(g)$ を属性 GUARDDEPCOUNT として用いる．

A.1.6 静的特徴の構成

定義 A.1.9 (静的特徴)．個々の比較ガードまたは比較命令について求める属性を a，その集計領域を R，集約方法を T とする．標的 P の静的特徴を

$$F_{a,R,T}(P) = T\big(\{a(x) \mid x \in R(P)\}\big)$$

と定義する．

本稿で用いる属性 a と適用する集計領域 R・集約方法 T を**表 A·1** に，集計領域と集約方法をそれぞれ表 A·2，**表 A·3** に示す．これらから，値が自明に定まる組合せを除いた 51 個の静的特徴を構成する．

表 A·1 において指示関数 $\mathbf{1}[\cdot]$ で表した属性が本文の二値属性に，それ以外が数値属性に対応する．本文の表 1 のとおり，HASHDEP は二値・数値の両方の用法を持つ．

なお，INPUTDIST に対する MEAN および MAX は，$d_{\mathrm{input}}(x) \geq 0$ を満たす比較のみを対象として求める．該当する比較が存在しない場合は値を 0 とする．

なお，HashConst は比較形式，HASHDEP はハッシュやチェックサム出力への依存を表すため，両者は別の属性として扱う．

また，INPUTDIST2 の閾値を 4 に変えた変種 INPUTDIST4 （$0 \leq d_{\mathrm{input}}(x) \leq 4$）も同じ規則で算出できるが，51 特徴には含めず，閾値の選び方に対する頑健性の確認にのみ用いた．閾値を 4 に変えても相関はほぼ変わらなかった（例：INPUTDIST2_CMP_FRAC の $\rho = -0.39$ に対し INPUTDIST4_CMP_FRAC は -0.38）．

表 A·1 属性 a と適用する集計領域・集約方法．
Table A·1 Attributes a and their applicable aggregation regions and operators.

記号	属性 a	適用する $R \times T$
MAGIC	$\mathbf{1}[\tau(g)$=Magic$]$	G・B_{far} × COUNT・FRAC
HASHCONST	$\mathbf{1}[\tau(g)$=HashConst$]$	G・B_{far} × COUNT・FRAC
OTHER	$\mathbf{1}[\tau(g)$=Other$]$	G・B_{far} × COUNT・FRAC
HASHDEP	$\mathbf{1}[\mathrm{hashdep}(g)>0]$	G・B_{far} × COUNT・FRAC
	$\mathrm{hashdep}(g)$	G・B_{far} × MEAN・MAX
INPUTDEP	$\mathbf{1}[d_{\mathrm{input}}(x) \geq 0]$	C × FRAC；G・B_{far} × COUNT・FRAC
INPUTDIST2	$\mathbf{1}[0 \leq d_{\mathrm{input}}(x) \leq 2]$	C × FRAC；G・B_{far} × COUNT・FRAC
INPUTDIST	$d_{\mathrm{input}}(x)$ (≥ 0 のみ)	C × MEAN；G・B_{far} × MEAN・MAX
WIDTH	$m(g)$	G・B_{far} × MEAN・MAX
GUARDDEP	$\mathbf{1}[\mathrm{guarddep}(g)>0]$	G・B_{far} × COUNT・FRAC
GUARDDEPCOUNT	$\mathrm{guarddep}(g)$	G・B_{far} × MEAN・MAX
ENTRYDIST	$d_{\mathrm{entry}}(g)$	B_{far} × MEAN・MAX
FARFROMENTRY	$\mathbf{1}[g \in B_{\mathrm{far}}]$	G × COUNT・FRAC

x は比較ガードまたは比較命令，C・G・B_{far} は**表 A·2** の集計領域を表す．HASHDEP は依存の有無（$\mathbf{1}[\mathrm{hashdep}(g)>0]$）を COUNT・FRAC で，個数（$\mathrm{hashdep}(g)$）を MEAN・MAX で集約する．

表 A·2 集計領域 R．
Table A·2 Aggregation regions R.

記号	領域
C（CMP）	全比較命令
G（GUARD）	全比較ガード
B_{far}（FAR）	入力処理入口から遠い比較ガード

表 A·3 集約方法 T．
Table A·3 Aggregation operators T.

演算	定義
COUNT	個数
FRAC	割合
MEAN	平均
MAX	最大

A.2 実装の詳細と抽出結果の確認

3 章で定義した静的特徴を LLVM 15 IR から抽出する解

© 2026 Information Processing Society of Japan

表 A·4 代表標的における比較ガードの抽出結果.

Table A·4 Extraction results of comparison guards for representative targets.

標的	全ガード	Magic	HashConst
jsoncpp	952	112	0
stb image	578	34	0
zlib	1692	188	0
libpng	3338	361	18
mbedtls	1262	240	42
curl	41656	4524	376
openssl	49067	3952	566

析器を Rust で実装した．解析器は `llvm-ir` クレートを用いてビットコードを読み込み，制御フロー，支配関係，および def-use 関係を構築する．

def-use 関係には SSA 上の def-use 辺に加え，同一の基底オブジェクトに対する store–load 辺を追加する．依存関係の探索は関数内で行い，関数呼出しの戻り値で打ち切る．

入力は関数引数に由来するポインタからの load，ハッシュやチェックサムの出力は，指定した関数名に該当する呼出しの戻り値として識別する．

ハッシュ・チェックサム関数の識別には，`crc32`, `adler32`, `md5`，`sha*`，`checksum`，`digest` 等の 22 個の名前パターンを用いた．

また，`LLVMFuzzerTestOneInput` の入口を入力処理入口とし，手続き間制御フローグラフ上で各比較ガードまでの ENTRYDIST を求める．最後に，各比較ガードまたは比較命令について取得した属性を標的単位に集約し，静的特徴を算出する．解析はコンパイラの改造やプログラムの実行を必要とせず，36 標的で合計 63.3 万個の比較ガードを解析した．

解析対象の IR は，FuzzBench の標的定義に従い，実測時と同一のハーネスおよび同一のビルド手順で標的をビルドして生成した．

表 A·4 に，代表的な標的における比較ガード数と Magic, HashConst の抽出件数を示す．

例えば，jsoncpp や stb では HashConst が検出されない一方，libpng では CRC，mbedtls や openssl では MAC や署名等に関連する処理中の比較が HashConst として検出された．

さらに，libpng の代表的な 3 つのチャンク処理についてソースコードと照合したところ，CRC に関する HashConst の分類は一致した．なお，これは限定的な確認であり，網羅的な精度評価ではない．

A.3 ファジング実験の設定とデータ

以下では，試行のランダム性の扱いと，公開データで欠損した実験条件の再取得について補足する．

ファジングのランダム性の扱い． 本稿および参照するデータでは，乱数のシードを固定して実行を再現可能にするのではなく，同一の条件で独立な試行を繰り返し，その分布どうしを比較することでランダム性の影響を扱う．試行間で共通に固定するのは，標的のビルド，ハーネスに与えるシードコーパス，時間予算，および 1 試行あたりの計算資源であり，これらを揃えたうえで残る差を乱数由来のばらつきとして扱う．

欠損した組の再取得． 公開データでは，実験条件と標的の組のうち 6 組の実測値が得られない．具体的には，bloaty と ngram8，openh264 と ngram4，openh264 と ngram8，harfbuzz と ngram8，harfbuzz と naive_ctx，systemd と naive_ctx の各組である．これらについては本稿でファジングを実施し，各組 20 試行を 23 時間ずつ実行した．標的は，公開データの収集時期に合わせたソースコードからビルドした．カバレッジは，実行終了時のコーパスを共通のカバレッジ計測用バイナリで再生し，llvm-cov が報告する被覆分岐数として求めた．

測定尺度の一致は，公開データに naive 条件が存在する bloaty で確認した（本稿の測定値平均 6098 に対し公開データ平均 6100）．一方，openh264，harfbuzz，systemd では，同じ naive 条件について本稿の測定値と公開データの平均にそれぞれ −112，+315，+3 被覆分岐数の差があり，公開データの naive 条件の標準偏差（14.7，29.0，1.1）を上回った．そこで，これら 3 標的の再取得した組では，性能差の基準にも本稿のパイプラインで取得した naive 条件（各 20 試行）を用いる．bloaty では尺度の一致が確認できるため，公開データの naive 条件を基準とする．

このほか，openh264 と naive_ctx の組では，公開データに 3 試行しか含まれない．ただし，同じ組の 19 試行が FuzzBench の実験データに存在するため，本稿ではこれを再集約して用いる．この 19 試行は naive 条件と同一の実験に含まれるため，この組に限り naive 条件との比較は実験をまたがない．

A.4 全静的特徴と CmpLog の性能差との相関一覧

表 A·5 に，3 章の構成規則から得た全 51 静的特徴と，CmpLog 条件の naive 条件に対する性能差との Spearman 順位相関係数を示す．ρ は，両ベンチマークを統合した 36 標的における相関係数である．また，ρ_{FB} と ρ_{SBFT} は，それぞれ FuzzBench の 23 標的と SBFT'23 の 13 標的における相関係数を表す．ρ_{part} は，比較ガード数を制御した Spearman の一次偏相関係数である．

A.5 技法横断分析の詳細

図 A·1 に，全 51 静的特徴と各実験条件の naive 条件に対する性能差との ρ を示す．

© 2026 Information Processing Society of Japan

表 A·5 全 51 静的特徴と CmpLog の有無による性能差との順位相関.

Table A·5 Spearman rank correlations between all 51 static features and the performance difference between configurations with and without CmpLog.

特徴	ρ	ρ_{FB}	ρ_{SBFT}	ρ_{part}	特徴	ρ	ρ_{FB}	ρ_{SBFT}	ρ_{part}
ENTRYDIST_FAR_MAX	+0.22	+0.33	+0.03	+0.14	INPUTDEP_FAR_FRAC	−0.20	−0.21	−0.07	−0.17
ENTRYDIST_FAR_MEAN	+0.16	+0.17	+0.31	+0.10	INPUTDEP_GUARD_COUNT	+0.18	+0.30	−0.09	+0.01
FARFROMENTRY_GUARD_COUNT	+0.14	+0.20	+0.02	+0.06	INPUTDEP_GUARD_FRAC	−0.33	−0.34	−0.10	−0.30
FARFROMENTRY_GUARD_FRAC	−0.04	+0.01	−0.17	+0.00	INPUTDIST2_CMP_FRAC	−0.39	−0.37	−0.01	−0.36
GUARDDEPCOUNT_FAR_MAX	−0.07	+0.08	−0.37	−0.10	INPUTDIST2_FAR_COUNT	+0.06	+0.08	+0.05	−0.02
GUARDDEPCOUNT_FAR_MEAN	−0.22	−0.12	−0.38	−0.20	INPUTDIST2_FAR_FRAC	−0.21	−0.30	−0.01	−0.20
GUARDDEPCOUNT_GUARD_MAX	+0.05	+0.16	−0.14	−0.08	INPUTDIST2_GUARD_COUNT	+0.17	+0.29	−0.14	−0.06
GUARDDEPCOUNT_GUARD_MEAN	−0.18	−0.06	−0.49	−0.20	INPUTDIST2_GUARD_FRAC	−0.36	−0.40	+0.01	−0.34
GUARDDEP_FAR_COUNT	0.00	+0.12	−0.23	−0.08	INPUTDIST_CMP_MEAN	−0.07	+0.02	−0.40	−0.05
GUARDDEP_FAR_FRAC	−0.23	−0.14	−0.41	−0.21	INPUTDIST_FAR_MAX	−0.10	−0.07	−0.10	−0.13
GUARDDEP_GUARD_COUNT	+0.13	+0.30	−0.23	−0.12	INPUTDIST_FAR_MEAN	+0.08	+0.19	−0.23	+0.09
GUARDDEP_GUARD_FRAC	−0.23	−0.07	−0.48	−0.21	INPUTDIST_GUARD_MAX	−0.07	−0.11	−0.03	−0.24
HASHCONST_FAR_COUNT	+0.50	+0.53	+0.33	+0.47	INPUTDIST_GUARD_MEAN	−0.07	+0.02	−0.40	−0.05
HASHCONST_FAR_FRAC	+0.51	+0.51	+0.39	+0.48	MAGIC_FAR_COUNT	+0.09	+0.18	−0.06	+0.00
HASHCONST_GUARD_COUNT	+0.54	+0.56	+0.44	+0.57	MAGIC_FAR_FRAC	+0.05	+0.09	+0.07	+0.01
HASHCONST_GUARD_FRAC	+0.51	+0.51	+0.53	+0.49	MAGIC_GUARD_COUNT	+0.20	+0.32	−0.12	+0.09
HASHDEP_FAR_COUNT	+0.41	+0.50	+0.13	+0.37	MAGIC_GUARD_FRAC	−0.05	−0.10	+0.09	−0.10
HASHDEP_FAR_FRAC	+0.43	+0.50	+0.21	+0.40	OTHER_FAR_COUNT	+0.13	+0.20	−0.02	+0.05
HASHDEP_FAR_MAX	+0.43	+0.50	+0.21	+0.39	OTHER_FAR_FRAC	−0.07	−0.05	−0.20	−0.06
HASHDEP_FAR_MEAN	+0.43	+0.50	+0.21	+0.39	OTHER_GUARD_COUNT	+0.19	+0.31	−0.20	+0.19
HASHDEP_GUARD_COUNT	+0.47	+0.56	+0.26	+0.46	OTHER_GUARD_FRAC	+0.09	+0.15	−0.18	+0.08
HASHDEP_GUARD_FRAC	+0.46	+0.55	+0.30	+0.43	WIDTH_FAR_MAX	+0.03	+0.23	−0.21	−0.03
HASHDEP_GUARD_MAX	+0.41	+0.56	+0.15	+0.39	WIDTH_FAR_MEAN	+0.10	+0.06	+0.21	+0.11
HASHDEP_GUARD_MEAN	+0.46	+0.54	+0.30	+0.44	WIDTH_GUARD_MAX	+0.15	+0.38	−0.17	+0.07
INPUTDEP_CMP_FRAC	−0.37	−0.36	−0.10	−0.34	WIDTH_GUARD_MEAN	+0.14	−0.01	+0.32	+0.12
INPUTDEP_FAR_COUNT	+0.06	+0.10	+0.02	−0.01					

© 2026 Information Processing Society of Japan

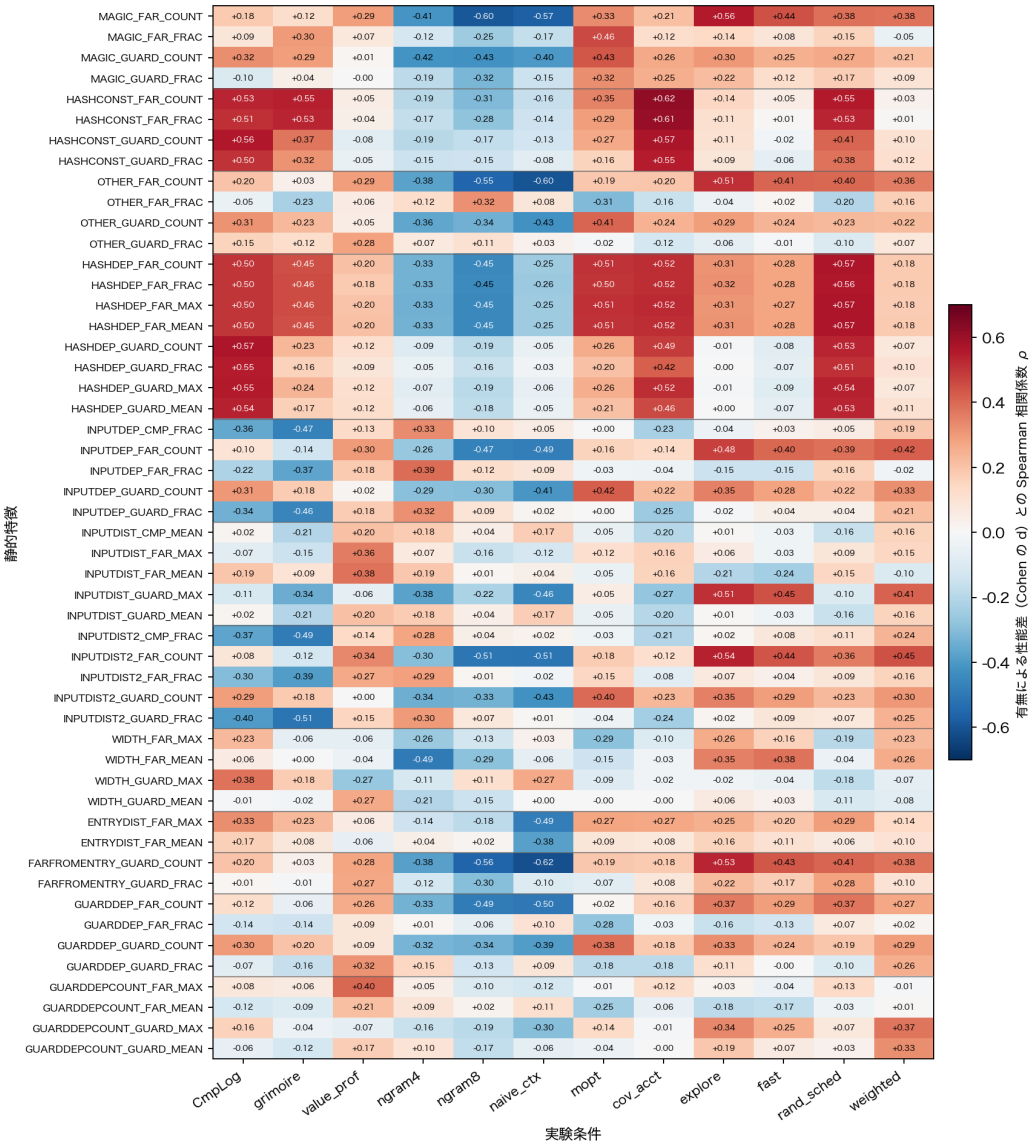


図 A·1 全 51 静的特徴と各実験条件の naive 条件に対する性能差との Spearman 順位相関係数（共通 23 標的）.

Fig. A·1 Spearman rank correlations between all 51 static features and the performance difference of each experimental condition relative to the naive condition (23 targets).

© 2026 Information Processing Society of Japan